
Les algorithmes de tri

Complexité • Correction • Terminaison

Diviser pour régner : tri fusion et tri rapide

CPGE — filières MP / PSI / TSI

Programme officiel d'informatique — réforme 2023

- ① Le problème du tri : vocabulaire et outils
- ② Les tris quadratiques : exercices de complexité
- ③ Une borne inférieure : $\Omega(n \log n)$
- ④ Diviser pour régner
- ⑤ Le tri fusion
- ⑥ Le tri rapide
- ⑦ Synthèse

Ce que dit le programme

Première année. Tri par insertion, tri par sélection (et tri à bulles) : mise en œuvre, *preuve de correction* par invariant de boucle, *terminaison* par variant, *coût* en nombre de comparaisons.

Deuxième année. Méthode **diviser pour régner** : tri fusion et tri rapide. Mise en équation du coût par une *relation de récurrence*, résolution, comparaison des complexités dans le meilleur cas, le pire cas et en moyenne. Notion de borne inférieure en $\Theta(n \log n)$ pour les tris par comparaisons.

Objectif de la séquence. On *réexploite* les tris de première année comme support d'exercices de *calcul de complexité* (le code est donné, la complexité est à établir), puis on construit les deux tris efficaces du programme de deuxième année.

Le problème du tri : vocabulaire et outils

- 1 Le problème du tri : vocabulaire et outils
- 2 Les tris quadratiques : exercices de complexité
- 3 Une borne inférieure : $\Omega(n \log n)$
- 4 Diviser pour régner
- 5 Le tri fusion
- 6 Le tri rapide
- 7 Synthèse

Problème du tri

Entrée : un tableau $t = [t[0], \dots, t[n-1]]$ de n éléments deux à deux comparables (relation d'ordre total $\leq$).

Sortie : une *permutation* σ de t telle que $t_{\sigma(0)} \leq t_{\sigma(1)} \leq \dots \leq t_{\sigma(n-1)}$.

Deux exigences, jamais une seule :

1. le tableau final est **trié** (croissant) ;

Problème du tri

Entrée : un tableau $t = [t[0], \dots, t[n-1]]$ de n éléments deux à deux comparables (relation d'ordre total $\leq$).

Sortie : une *permutation* σ de t telle que $t_{\sigma(0)} \leq t_{\sigma(1)} \leq \dots \leq t_{\sigma(n-1)}$.

Deux exigences, jamais une seule :

1. le tableau final est **trié** (croissant) ;
2. le tableau final est une **permutation** du tableau initial (on n'a ni perdu ni inventé d'élément).

Problème du tri

Entrée : un tableau $t = [t[0], \dots, t[n-1]]$ de n éléments deux à deux comparables (relation d'ordre total $\leq$).

Sortie : une *permutation* σ de t telle que $t_{\sigma(0)} \leq t_{\sigma(1)} \leq \dots \leq t_{\sigma(n-1)}$.

Deux exigences, jamais une seule :

1. le tableau final est **trié** (croissant) ;
2. le tableau final est une **permutation** du tableau initial (on n'a ni perdu ni inventé d'élément).

Oublier le point 2 est l'erreur classique : `return [1]*n` produit un tableau parfaitement trié... et faux.

- **Opération élémentaire de référence** : la **comparaison** entre deux éléments. On compte à part les **échanges** et les **décalages**.
- **Tri en place** : la mémoire auxiliaire utilisée est $\mathcal{O}(1)$ ou $\mathcal{O}(\log n)$ (on ne recopie pas le tableau).
- **Tri stable** : deux éléments de même clé conservent leur ordre relatif.
- **Inversion** de t : couple (i, j) avec $i < j$ et $t[i] > t[j]$. On note $I(t)$ leur nombre.

$$0 \leq I(t) \leq \binom{n}{2} = \frac{n(n-1)}{2}, \text{ avec :}$$

- $I(t) = 0 \iff t$ est déjà trié ;
- $I(t) = \binom{n}{2} \iff t$ est trié en ordre strictement décroissant.

Un échange d'éléments voisins fait varier I de exactement ± 1 : c'est la clé du coût du tri par insertion et du tri à bulles.

On note $C(n)$ le nombre de comparaisons effectuées sur une entrée de taille n .

- **Pire cas** : $C_{\max}(n) = \max\{C(t) ; |t| = n\}$ — la garantie.
- **Meilleur cas** : $C_{\min}(n)$ — ce qu'on ne peut pas battre.
- **Cas moyen** : $C_{\text{moy}}(n) = \frac{1}{n!} \sum_{\sigma \in \mathfrak{S}_n} C(\sigma)$, sous l'hypothèse d'équiprobabilité des $n!$ permutations.

$f = \Theta(g)$ signifie : $\exists c_1, c_2 > 0, \exists n_0, \forall n \geq n_0, c_1 g(n) \leq f(n) \leq c_2 g(n)$.

Sommes utiles : $\sum_{k=1}^n k = \frac{n(n+1)}{2}, \quad H_n = \sum_{k=1}^n \frac{1}{k} = \ln n + \gamma + o(1), \quad n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n.$

Sauf mention contraire, $\log$ désigne le logarithme en base 2.

Les tris quadratiques : exercices de complexité

- 1 Le problème du tri : vocabulaire et outils
- 2 Les tris quadratiques : exercices de complexité**
- 3 Une borne inférieure : $\Omega(n \log n)$
- 4 Diviser pour régner
- 5 Le tri fusion
- 6 Le tri rapide
- 7 Synthèse

Tri par sélection

on cherche le minimum
et on le met devant

Tri par insertion

on insère chaque élément
à sa place dans le début trié

Tri à bulles

on échange les voisins
mal ordonnés, en boucle

Dans cette partie, le **code est donné**. Le travail demandé est celui de l'évaluateur de concours :

1. compter *exactement* les opérations élémentaires ;
2. en déduire la classe Θ dans le meilleur, le pire et le cas moyen ;
3. justifier *terminaison* (variant) et *correction* (invariant).

```
1 def tri_selection(t):
2     n = len(t)
3     for i in range(n - 1):          # boucle externe
4         imin = i
5         for j in range(i + 1, n):  # boucle interne
6             if t[j] < t[imin]:      # <-- comparaison comptee
7                 imin = j
8         t[i], t[imin] = t[imin], t[i]
9     return t
```

Principe. À l'étape i , la tranche $t[0..i - 1]$ est définitivement en place ; on cherche le minimum de $t[i..n - 1]$ et on l'échange avec $t[i]$.

Exercice 1 — comptage exact

1. Donner le nombre *exact* de comparaisons $t[j] < t[\text{imin}]$.
2. Dépend-il du contenu de t ? Conclure sur $C_{\min}$, $C_{\max}$, C_{moy} .
3. Donner le nombre exact d'échanges.

Exercice 1 — comptage exact

1. Donner le nombre *exact* de comparaisons $t[j] < t[\text{imin}]$.
2. Dépend-il du contenu de t ? Conclure sur $C_{\min}$, $C_{\max}$, C_{moy} .
3. Donner le nombre exact d'échanges.

Solution

Pour i fixé, la boucle interne parcourt $j \in \{i + 1, \dots, n - 1\}$, soit $n - 1 - i$ comparaisons.

$$C(n) = \sum_{i=0}^{n-2} (n - 1 - i) \stackrel{k=n-1-i}{=} \sum_{k=1}^{n-1} k = \frac{n(n-1)}{2}.$$

Aucun test ne conditionne le *nombre* de tours : le comptage est **indépendant des données**. Donc $C_{\min}(n) = C_{\max}(n) = C_{\text{moy}}(n) = \frac{n(n-1)}{2} = \Theta(n^2)$.

Échanges : exactement $n - 1$, soit $\Theta(n)$ — c'est le *seul* avantage de ce tri (coût d'écriture minimal).

Terminaison — variant de boucle

Boucle interne : variant $v_j = n - j$, entier, ≥ 0 , strictement décroissant (car j augmente de 1) : elle s'arrête.

Boucle externe : variant $v_i = n - 1 - i$.

Les deux boucles étant `for` sur des intervalles finis, la terminaison est immédiate ; l'exercice est de savoir l'écrire avec un variant.

Correction — invariant de boucle

Invariant $\mathcal{I}_i$: avant l'itération i , la tranche $t[0..i-1]$ est triée par ordre croissant et contient les i plus petits éléments du tableau initial (au sens du multi-ensemble), et t est une permutation du tableau initial.

- *Initialisation* ($i = 0$) : la tranche est vide, l'invariant est vrai.
- *Hérédité* : l'itération place en $t[i]$ le minimum de $t[i..n-1]$, lequel est $\geq \max t[0..i-1]$ par $\mathcal{I}_i$; seul un échange est effectué, donc t reste une permutation. D'où $\mathcal{I}_{i+1}$.
- *Sortie* ($i = n-1$) : $t[0..n-2]$ est trié et contient les $n-1$ plus petits ; $t[n-1]$ est donc le plus grand : t est trié.

Tri par insertion — le code

```
1 def tri_insertion(t):
2     n = len(t)
3     for i in range(1, n):
4         x = t[i]                # element a inserer
5         j = i - 1
6         while j >= 0 and t[j] > x: # <-- comparaison comptee
7             t[j + 1] = t[j]      # <-- decalage
8             j = j - 1
9         t[j + 1] = x
10    return t
```

Principe. C'est le tri du joueur de cartes : $t[0..i-1]$ est déjà trié, on fait « glisser » $x = t[i]$ vers la gauche jusqu'à sa place.

Exercice 2 — les deux extrêmes

1. Quelle entrée minimise le nombre de comparaisons ? Le calculer exactement.
2. Quelle entrée le maximise ? Le calculer exactement.
3. En déduire $C_{\min}(n)$ et $C_{\max}(n)$ en Θ .

Exercice 2 — les deux extrêmes

1. Quelle entrée minimise le nombre de comparaisons ? Le calculer exactement.
2. Quelle entrée le maximise ? Le calculer exactement.
3. En déduire $C_{\min}(n)$ et $C_{\max}(n)$ en Θ .

Solution

Meilleur cas : t déjà trié. Le test $t[j] > x$ est faux au premier essai pour chaque i : une seule comparaison par tour, soit $C_{\min}(n) = n - 1 = \Theta(n)$. *Le tri par insertion est linéaire sur une entrée presque triée* : c'est sa qualité remarquable.

Pire cas : t trié strictement décroissant. x descend jusqu'à l'indice 0 : i comparaisons au tour i , donc

$$C_{\max}(n) = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = \Theta(n^2).$$

Exercice 3 — décalages et inversions

Montrer que le nombre total de décalages $t[j+1] = t[j]$ est exactement $I(t)$, le nombre d'inversions du tableau initial. En déduire le coût moyen.

Exercice 3 — décalages et inversions

Montrer que le nombre total de décalages $\tau[j+1] = \tau[j]$ est exactement $I(t)$, le nombre d'inversions du tableau initial. En déduire le coût moyen.

Solution

Au tour i , on décale un élément $t[j]$ exactement lorsque $t[j] > x = t[i]$ avec $j < i$: c'est précisément une inversion (j, i) . Comme la tranche $t[0..i-1]$ contient les mêmes éléments que la tranche initiale, le nombre de décalages du tour i est le nombre d'inversions dont le second indice est i . En sommant : **nombre de décalages** = $I(t)$, et le nombre de comparaisons vaut $I(t) +$ (au plus $n - 1$).

En moyenne : pour une permutation aléatoire uniforme, chaque couple (i, j) , $i < j$, est une inversion avec probabilité $\frac{1}{2}$. Par linéarité de l'espérance,

$$\mathbb{E}[I] = \frac{1}{2} \binom{n}{2} = \frac{n(n-1)}{4}, \quad \text{donc} \quad C_{\text{moy}}(n) \sim \frac{n^2}{4} = \Theta(n^2).$$

Le cas moyen est quadratique, mais avec une constante deux fois meilleure que le pire cas.

Terminaison — variant de la boucle while

$v = j + 1$ est un entier, minoré par 0 grâce à la condition $j \geq 0$, et strictement décroissant à chaque tour ($j = j - 1$). La boucle `while` termine ; la boucle `for` est finie.

Correction — invariant de boucle

Invariant externe $\mathcal{I}_i$: avant l'itération i , $t[0..i - 1]$ est trié et t est une permutation du tableau initial.

Invariant interne : $t[j + 2..i]$ contient les éléments de $t[j + 1..i - 1]$ initiaux, tous $> x$, rangés dans le même ordre ; la case $t[j + 1]$ est libre.

- *Initialisation* : $i = 1$, tranche $t[0..0]$ triée.
- *Hérédité* : à la sortie du `while`, $j = -1$ ou $t[j] \leq x$; l'affectation $t[j+1] = x$ place x après tous les éléments $\leq x$ et avant tous les éléments $> x$: $t[0..i]$ est trié.
- *Sortie* : $i = n$, donc $t[0..n - 1]$ est trié.

Bonus : l'invariant interne montre aussi que le tri est **stable** (le test est $>$ strict, un élément égal n'est pas dépassé).

```
1 def tri_bulles(t):
2     n = len(t)
3     for i in range(n - 1, 0, -1):
4         echange = False
5         for j in range(i):
6             if t[j] > t[j + 1]:          # <-- comparaison
7                 t[j], t[j+1] = t[j+1], t[j] # <-- echange de voisins
8                 echange = True
9         if not echange:                  # optimisation : deja trie
10             return t
11     return t
```

Principe. À chaque passe, le maximum de la zone non triée « remonte » en fin de zone comme une bulle.

Exercice 4 — avec et sans l'optimisation

1. Sans le drapeau échange, combien de comparaisons exactement ?
2. Avec le drapeau, que vaut $C_{\min}(n)$? Sur quelle entrée ?
3. Montrer que le nombre d'échanges vaut exactement $I(t)$.

Exercice 4 — avec et sans l'optimisation

1. Sans le drapeau échange, combien de comparaisons exactement ?
2. Avec le drapeau, que vaut $C_{\min}(n)$? Sur quelle entrée ?
3. Montrer que le nombre d'échanges vaut exactement $I(t)$.

Solution

1. Pour i de $n - 1$ à 1, la boucle interne fait i comparaisons : $\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = \Theta(n^2)$, quelle que soit l'entrée.
2. Si t est déjà trié, la première passe ne provoque aucun échange et on sort : $C_{\min}(n) = n - 1 = \Theta(n)$. Le pire cas reste $\frac{n(n-1)}{2} = \Theta(n^2)$ (entrée décroissante).
3. Un échange porte sur deux voisins mal ordonnés : il supprime exactement une inversion et n'en crée aucune (l'ordre relatif des autres couples est inchangé). Comme on part de $I(t)$ inversions et qu'on termine à 0, le nombre d'échanges est exactement $I(t)$; en moyenne $\frac{n(n-1)}{4}$.

Terminaison — variant de boucle

Boucle externe : variant $v_i = i$, entier positif strictement décroissant. Boucle interne : variant $i - j$.

Correction — invariant de boucle

Invariant de la boucle interne : après le traitement de l'indice j , $t[j+1] = \max(t[0..j+1])$ — le maximum courant est « poussé » vers la droite.

Invariant de la boucle externe $\mathcal{I}_i$: la tranche $t[i+1..n-1]$ est triée, contient les $n-1-i$ plus grands éléments, et t est une permutation de l'entrée.

- *Initialisation* : $i = n-1$, tranche vide.
- *Hérédité* : la passe amène $\max t[0..i]$ en position i , et n'utilise que des échanges (permutation préservée).
- *Sortie* : $i = 0$ donne $t[1..n-1]$ trié et $t[0]$ minimal, donc t trié.
- *Sortie anticipée* : échange faux signifie $t[j] \leq t[j+1]$ pour tout $j < i$, donc $t[0..i]$ est trié ; avec $\mathcal{I}_i$, t est trié.

Tri	$C_{\min}$	$C_{\max}$	C_{moy}	Échanges	En place	Stable
Sélection	$\frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$	$n - 1$	oui	non
Insertion	$n - 1$	$\frac{n(n-1)}{2}$	$\sim \frac{n^2}{4}$	$I(t)$ décal.	oui	oui
Bulles	$n - 1$	$\frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$	$I(t)$	oui	oui

Les trois sont en $\Theta(n^2)$ dans le pire cas. Pour $n = 10^6$, cela représente $\approx 5 \cdot 10^{11}$ comparaisons : plusieurs heures. Un tri en $\Theta(n \log n)$ en demande $\approx 2 \cdot 10^7$: une fraction de seconde.

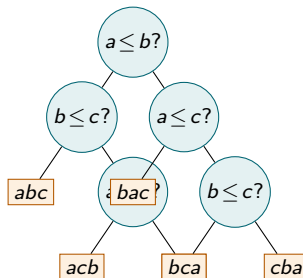
Question : peut-on faire mieux que n^2 ? Et jusqu'où ?

Une borne inférieure : $\Omega(n \log n)$

- 1 Le problème du tri : vocabulaire et outils
- 2 Les tris quadratiques : exercices de complexité
- 3 Une borne inférieure : $\Omega(n \log n)$**
- 4 Diviser pour régner
- 5 Le tri fusion
- 6 Le tri rapide
- 7 Synthèse

L'arbre de décision

Un tri *par comparaisons* n'accède aux données que par des tests $t[i] \leq t[j]$. Son exécution sur les $n!$ entrées possibles se représente par un arbre binaire :



- chaque *nœud interne* = une comparaison, ses deux fils = les deux réponses ;
- chaque *feuille* = une permutation produite en sortie ;
- le nombre de comparaisons sur une entrée = la profondeur de la feuille atteinte ;
- $C_{\max}(n)$ = la **hauteur** h de l'arbre.

Le théorème de la borne inférieure

Théorème

Tout algorithme de tri par comparaisons effectuée, dans le pire cas, au moins $\lceil \log_2(n!) \rceil = \Omega(n \log n)$ comparaisons.

Théorème

Tout algorithme de tri par comparaisons effectuée, dans le pire cas, au moins $\lceil \log_2(n!) \rceil = \Omega(n \log n)$ comparaisons.

Démonstration. L'algorithme doit pouvoir produire les $n!$ permutations : l'arbre possède au moins $n!$ feuilles. Or un arbre binaire de hauteur h a au plus 2^h feuilles. Donc

$$2^h \geq n! \quad \implies \quad h \geq \log_2(n!).$$

Par la formule de Stirling, $n! \geq \left(\frac{n}{e}\right)^n$, d'où

$$h \geq n \log_2 n - n \log_2 e = \Theta(n \log n). \quad \blacksquare$$

Le théorème de la borne inférieure

Théorème

Tout algorithme de tri par comparaisons effectuée, dans le pire cas, au moins $\lceil \log_2(n!) \rceil = \Omega(n \log n)$ comparaisons.

Démonstration. L'algorithme doit pouvoir produire les $n!$ permutations : l'arbre possède au moins $n!$ feuilles. Or un arbre binaire de hauteur h a au plus 2^h feuilles. Donc

$$2^h \geq n! \quad \implies \quad h \geq \log_2(n!).$$

Par la formule de Stirling, $n! \geq \left(\frac{n}{e}\right)^n$, d'où

$$h \geq n \log_2 n - n \log_2 e = \Theta(n \log n). \quad \blacksquare$$

Conséquence : $\Theta(n \log n)$ est **optimal** pour un tri par comparaisons. Les tris fusion et rapide atteignent cette borne — le second seulement en moyenne.

Diviser pour régner

- 1 Le problème du tri : vocabulaire et outils
- 2 Les tris quadratiques : exercices de complexité
- 3 Une borne inférieure : $\Omega(n \log n)$
- 4 Diviser pour régner**
- 5 Le tri fusion
- 6 Le tri rapide
- 7 Synthèse

Diviser pour régner (*divide and conquer*)

Diviser découper le problème de taille n en a sous-problèmes de taille n/b ;

Régner les résoudre *récurivement* (cas de base : taille ≤ 1) ;

Combiner reconstruire la solution globale à partir des solutions partielles.

Le coût obéit alors à une **relation de récurrence** :

$$T(n) = a T\left(\frac{n}{b}\right) + f(n), \quad T(1) = \Theta(1),$$

où $f(n)$ est le coût de la division et de la combinaison.

Diviser		Combiner	
Tri fusion	trivial (milieu)	coûteux (fusion en $\Theta(n)$)	$T(n) = 2T(n/2) + \Theta(n)$
Tri rapide	coûteux (partition $\Theta(n)$)	trivial (rien)	$T(n) = T(k) + T(n-1-k) + \Theta(n)$

Les deux tris sont duaux l'un de l'autre.

Le théorème maître (*master theorem*)

Soit $T(n) = a T(n/b) + \Theta(n^c)$ avec $a \geq 1$, $b > 1$, $c \geq 0$. Posons $c^* = \log_b a$. Alors :

$$T(n) = \begin{cases} \Theta(n^c) & \text{si } c > c^* \quad (\text{la combinaison domine}), \\ \Theta(n^c \log n) & \text{si } c = c^* \quad (\text{équilibre}), \\ \Theta(n^{c^*}) & \text{si } c < c^* \quad (\text{les feuilles dominent}). \end{cases}$$

Exercice 5 — application immédiate

Appliquer le théorème à $T(n) = 2T(n/2) + \Theta(n)$, puis à $T(n) = 2T(n/2) + \Theta(1)$.

Le théorème maître (*master theorem*)

Soit $T(n) = a T(n/b) + \Theta(n^c)$ avec $a \geq 1$, $b > 1$, $c \geq 0$. Posons $c^* = \log_b a$. Alors :

$$T(n) = \begin{cases} \Theta(n^c) & \text{si } c > c^* \quad (\text{la combinaison domine}), \\ \Theta(n^c \log n) & \text{si } c = c^* \quad (\text{équilibre}), \\ \Theta(n^{c^*}) & \text{si } c < c^* \quad (\text{les feuilles dominent}). \end{cases}$$

Exercice 5 — application immédiate

Appliquer le théorème à $T(n) = 2T(n/2) + \Theta(n)$, puis à $T(n) = 2T(n/2) + \Theta(1)$.

Solution

Premier cas : $a = b = 2$, $c = 1$, $c^* = \log_2 2 = 1$, donc $c = c^*$: $T(n) = \Theta(n \log n)$ — c'est le **tri fusion**.

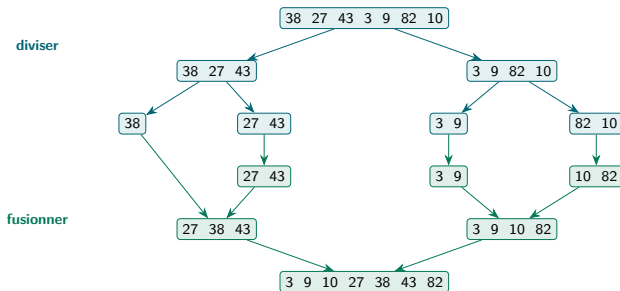
Second cas : $c = 0 < c^* = 1$, donc $T(n) = \Theta(n)$ (exemple : compter les feuilles d'un arbre).

Le tri fusion

- 1 Le problème du tri : vocabulaire et outils
- 2 Les tris quadratiques : exercices de complexité
- 3 Une borne inférieure : $\Omega(n \log n)$
- 4 Diviser pour régner
- 5 Le tri fusion**
- 6 Le tri rapide
- 7 Synthèse

Idée

Fusionner deux listes *déjà triées* est facile et linéaire : on compare les deux têtes et on prend la plus petite. D'où l'algorithme : couper en deux, trier chaque moitié récursivement, fusionner.



```
1 def fusion(u, v):
2     """u et v sont deux listes triees ; renvoie la liste triee des elements."""
3     res = []
4     i, j = 0, 0
5     while i < len(u) and j < len(v):
6         if u[i] <= v[j]:          # <= : garantit la stabilite
7             res.append(u[i]); i += 1
8         else:
9             res.append(v[j]); j += 1
10    return res + u[i:] + v[j:]    # l'une des deux tranches est vide
```

Terminaison

Variant $v = (|u| - i) + (|v| - j)$: entier ≥ 0 , décroît strictement de 1 à chaque tour.

Correction — invariant de boucle

Invariant : *res* est trié, *res* contient exactement $u[0..i-1]$ et $v[0..j-1]$, et tout élément de *res* est $\leq$ tout élément de $u[i..]$ et de $v[j..]$.

- *Initialisation* : *res* vide, propriétés vraies.
- *Hérédité* : on ajoute $\min(u[i], v[j])$; comme u et v sont triées, cet élément minore tout le reste, et il est $\geq$ le dernier de *res*.
- *Sortie* : une des deux listes est épuisée ; la concaténation du reste, déjà trié et plus grand que tout *res*, conserve l'invariant.

Coût. Chaque tour de boucle ajoute un élément à *res* : au plus $|u| + |v|$ tours, chacun à une comparaison. Donc

$$(\min(|u|, |v|)) \leq C_{\text{fusion}} \leq |u| + |v| - 1 = \Theta(n).$$

Le tri fusion

```
1 def tri_fusion(t):
2     n = len(t)
3     if n <= 1:                # cas de base
4         return t
5     m = n // 2                # diviser
6     gauche = tri_fusion(t[:m]) # regner
7     droite = tri_fusion(t[m:])
8     return fusion(gauche, droite) # combiner
```

Terminaison — fonction de mesure

$\mu(t) = |t| \in \mathbb{N}$. Si $n \geq 2$ alors $1 \leq m \leq n - 1$, donc $\mu(t[:m]) = m < n$ et $\mu(t[m:]) = n - m < n$: la mesure décroît strictement à chaque appel, et le cas de base $n \leq 1$ est atteint. La récursion est *bien fondée*.

Correction — récurrence forte sur n

Base : $n \leq 1$, t est trié. *Hérédité* : si `tri_fusion` trie correctement toute liste de taille $< n$, alors *gauche* et *droite* sont triées et sont des permutations des deux moitiés ; la fusion (correcte, prouvée ci-dessus) renvoie une liste triée qui est une permutation de t .

Exercice 6 — mise en équation et résolution

1. Écrire la récurrence vérifiée par $C(n)$, nombre de comparaisons dans le pire cas.
2. La résoudre pour $n = 2^p$ en déroulant, puis conclure par le théorème maître.
3. Le pire cas et le meilleur cas sont-ils du même ordre ?

Exercice 6 — mise en équation et résolution

1. Écrire la récurrence vérifiée par $C(n)$, nombre de comparaisons dans le pire cas.
2. La résoudre pour $n = 2^p$ en déroulant, puis conclure par le théorème maître.
3. Le pire cas et le meilleur cas sont-ils du même ordre ?

Solution

1. $C(n) = C(\lceil n/2 \rceil) + C(\lfloor n/2 \rfloor) + (n - 1)$, $C(1) = 0$.
2. Pour $n = 2^p$: $C(n) = 2 C(n/2) + n - 1$. En divisant par n et en posant $u_p = C(2^p)/2^p$:

$$u_p = u_{p-1} + 1 - 2^{-p} \implies u_p = p - \sum_{k=1}^p 2^{-k} = p - 1 + 2^{-p}.$$

Donc $C(n) = n \log_2 n - n + 1$. Théorème maître ($a = b = 2$, $c = 1 = c^*$) : $C(n) = \Theta(n \log n)$.

3. Oui : le nombre de tours de fusion ne dépend quasiment pas des données ($\geq \frac{n}{2}$ comparaisons par fusion). On a $C_{\min}(n) = C_{\max}(n) = C_{\text{moy}}(n) = \Theta(n \log n)$: **aucune entrée ne dégrade le tri fusion.**

Tri fusion — lecture de l'arbre d'appels



$$\log_2 n \text{ niveaux} \times \text{un coût de } \Theta(n) \text{ par niveau} = \Theta(n \log n).$$

Meilleur / moyen / pire cas	$\Theta(n \log n)$ dans les trois cas
Comparaisons exactes ($n = 2^p$, pire cas)	$n \log_2 n - n + 1$
Mémoire auxiliaire	$\Theta(n)$ — n'est pas en place
Stabilité	stable (grâce au test $u[i] \leq v[j]$)
Profondeur de récursion	$\Theta(\log n)$

Atouts. Complexité garantie, stabilité, comportement idéal sur les données lues séquentiellement (fichiers, listes chaînées) : c'est le tri des tris externes.

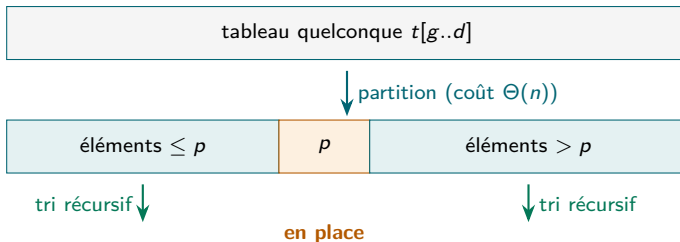
Limite. Le coût mémoire $\Theta(n)$ (et les recopies $t[:m]$, $t[m:]$) le rendent plus lent en pratique qu'un tri rapide bien implanté sur un tableau en mémoire.

Le tri rapide

- 1 Le problème du tri : vocabulaire et outils
- 2 Les tris quadratiques : exercices de complexité
- 3 Une borne inférieure : $\Omega(n \log n)$
- 4 Diviser pour régner
- 5 Le tri fusion
- 6 Le tri rapide**
- 7 Synthèse

Idée

On choisit un élément p , le **pivot**, et on **partitionne** le tableau : à gauche les éléments $\leq p$, à droite les éléments $> p$. Le pivot est alors *définitivement à sa place*. Il ne reste qu'à trier récursivement les deux côtés — **sans rien recombinaer**.



La partition (schéma de Lomuto)

```
1 def partition(t, g, d):
2     """Partitionne t[g..d] autour du pivot t[d] ; renvoie l'indice final du pivot."""
3     p = t[d]
4     i = g # frontiere : t[g..i-1] contient les elements <= p
5     for j in range(g, d):
6         if t[j] <= p: # <-- comparaison comptee
7             t[i], t[j] = t[j], t[i]
8             i += 1
9     t[i], t[d] = t[d], t[i] # on place le pivot
10    return i
```

Correction — invariant de la partition

Avant le tour j : $t[g..i-1]$ ne contient que des éléments $\leq p$, et $t[i..j-1]$ que des éléments $> p$.
Initialisation : les deux tranches sont vides. Hérédité : si $t[j] \leq p$ on l'échange avec le premier élément $> p$ et on avance la frontière ; sinon la tranche des « $> p$ » s'allonge. Sortie ($j = d$) : l'échange final place p entre les deux tranches, donc à sa place définitive.

```
1 def tri_rapide(t, g=0, d=None):
2     if d is None:
3         d = len(t) - 1
4     if g < d:
5         k = partition(t, g, d) # diviser (le travail est ici)
6         tri_rapide(t, g, k - 1) # regner
7         tri_rapide(t, k + 1, d)
8     return t
```

Terminaison — fonction de mesure

$\mu(g, d) = d - g + 1$ (taille de la tranche), entier ≥ 0 . Comme $g \leq k \leq d$, les deux appels portent sur des tranches de tailles $k - g$ et $d - k$, toutes deux $\leq \mu - 1 < \mu$: la mesure décroît strictement. *C'est le fait que le pivot soit exclu des deux appels qui assure la terminaison.*

Correction — récurrence forte sur la taille de la tranche

Base : tranche de taille ≤ 1 , triée. Hérédité : après partition, $t[k]$ est à sa place, tout $t[g..k - 1]$ est $\leq t[k] \leq$ tout $t[k + 1..d]$; par hypothèse de récurrence les deux tranches sont triées, donc $t[g..d]$ l'est.

Exercice 7 — la partition dégénérée

1. Sur quelle entrée le pivot $t[d]$ est-il systématiquement le maximum de la tranche ? Que deviennent alors les deux appels récurifs ?
2. En déduire la récurrence, puis $C_{\max}(n)$.
3. Quelle est la profondeur de récursion dans ce cas ?

Exercice 7 — la partition dégénérée

1. Sur quelle entrée le pivot $t[d]$ est-il systématiquement le maximum de la tranche ? Que deviennent alors les deux appels récurifs ?
2. En déduire la récurrence, puis $C_{\max}(n)$.
3. Quelle est la profondeur de récursion dans ce cas ?

Solution

1. Si t est **déjà trié** (ou trié à l'envers), le pivot est un extremum : la partition renvoie $k = d$ (resp. $k = g$), donc une tranche de taille $n - 1$ et une tranche vide.
2. $C(n) = C(n - 1) + (n - 1)$, $C(0) = C(1) = 0$, d'où

$$C(n) = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = \Theta(n^2).$$

3. Profondeur n : sur $n = 10^5$, Python lève `RecursionError`.

Ironie : le tri rapide est le plus lent des tris sur une entrée déjà triée.

Exercice 8 — la partition équilibrée

Supposer qu'à chaque appel le pivot tombe au milieu. Écrire et résoudre la récurrence.

Exercice 8 — la partition équilibrée

Supposer qu'à chaque appel le pivot tombe au milieu. Écrire et résoudre la récurrence.

Solution

$C(n) = 2 C\left(\frac{n-1}{2}\right) + (n-1) \leq 2 C(n/2) + n$. Théorème maître avec $a = b = 2$, $c = 1 = c^*$:

$$C_{\min}(n) = \Theta(n \log n),$$

et la profondeur de récursion tombe à $\Theta(\log n)$.

Exercice 8 — la partition équilibrée

Supposer qu'à chaque appel le pivot tombe au milieu. Écrire et résoudre la récurrence.

Solution

$C(n) = 2 C\left(\frac{n-1}{2}\right) + (n-1) \leq 2 C(n/2) + n$. Théorème maître avec $a = b = 2$, $c = 1 = c^*$:

$$C_{\min}(n) = \Theta(n \log n),$$

et la profondeur de récursion tombe à $\Theta(\log n)$.

Le grand écart. $\Theta(n \log n)$ dans le meilleur cas, $\Theta(n^2)$ dans le pire : tout dépend de la qualité du pivot. La question décisive est donc : **que se passe-t-il en moyenne ?**

On suppose les $n!$ permutations équiprobables ; le pivot est alors, avec probabilité $\frac{1}{n}$, le $(k + 1)$ -ième plus petit élément, pour chaque $k \in \{0, \dots, n - 1\}$.

$$C(n) = (n - 1) + \frac{1}{n} \sum_{k=0}^{n-1} (C(k) + C(n - 1 - k)) = (n - 1) + \frac{2}{n} \sum_{k=0}^{n-1} C(k).$$

On suppose les $n!$ permutations équiprobables ; le pivot est alors, avec probabilité $\frac{1}{n}$, le $(k+1)$ -ième plus petit élément, pour chaque $k \in \{0, \dots, n-1\}$.

$$C(n) = (n-1) + \frac{1}{n} \sum_{k=0}^{n-1} (C(k) + C(n-1-k)) = (n-1) + \frac{2}{n} \sum_{k=0}^{n-1} C(k).$$

Résolution. On multiplie par n puis on soustrait la relation au rang $n-1$:

$$nC(n) - (n-1)C(n-1) = n(n-1) - (n-1)(n-2) + 2C(n-1) = 2(n-1) + 2C(n-1),$$

$$\text{d'où} \quad nC(n) = (n+1)C(n-1) + 2(n-1) \quad \implies \quad \frac{C(n)}{n+1} = \frac{C(n-1)}{n} + \frac{2(n-1)}{n(n+1)}.$$

On suppose les $n!$ permutations équiprobables ; le pivot est alors, avec probabilité $\frac{1}{n}$, le $(k+1)$ -ième plus petit élément, pour chaque $k \in \{0, \dots, n-1\}$.

$$C(n) = (n-1) + \frac{1}{n} \sum_{k=0}^{n-1} (C(k) + C(n-1-k)) = (n-1) + \frac{2}{n} \sum_{k=0}^{n-1} C(k).$$

Résolution. On multiplie par n puis on soustrait la relation au rang $n-1$:

$$nC(n) - (n-1)C(n-1) = n(n-1) - (n-1)(n-2) + 2C(n-1) = 2(n-1) + 2C(n-1),$$

$$\text{d'où} \quad nC(n) = (n+1)C(n-1) + 2(n-1) \quad \implies \quad \frac{C(n)}{n+1} = \frac{C(n-1)}{n} + \frac{2(n-1)}{n(n+1)}.$$

En posant $w_n = \frac{C(n)}{n+1}$ et en télescopant, puis en utilisant $\frac{2(k-1)}{k(k+1)} \leq \frac{2}{k}$:

$$w_n = \sum_{k=1}^n \frac{2(k-1)}{k(k+1)} = 2H_{n+1} - \frac{4n}{n+1} \quad \implies \quad \boxed{C_{\text{moy}}(n) = 2(n+1)H_{n+1} - 4n}$$

Avec $H_n = \ln n + \gamma + o(1)$:

$$C_{\text{moy}}(n) \sim 2n \ln n = \frac{2}{\log_2 e} n \log_2 n \approx 1,39 n \log_2 n = \Theta(n \log n).$$

En moyenne, le tri rapide n'est qu'environ **39 % plus coûteux en comparaisons** que l'optimum théorique $n \log_2 n$ — et il travaille **en place**, sans recopie, avec une excellente localité mémoire. C'est pourquoi il est, en pratique, le plus rapide.

Avec $H_n = \ln n + \gamma + o(1)$:

$$C_{\text{moy}}(n) \sim 2n \ln n = \frac{2}{\log_2 e} n \log_2 n \approx 1,39 n \log_2 n = \Theta(n \log n).$$

En moyenne, le tri rapide n'est qu'environ **39 % plus coûteux en comparaisons** que l'optimum théorique $n \log_2 n$ — et il travaille **en place**, sans recopie, avec une excellente localité mémoire. C'est pourquoi il est, en pratique, le plus rapide.

Comment éviter le pire cas ?

- **Pivot aléatoire** : $i = \text{random.randint}(g, d)$ puis échange avec $t[d]$. Le pire cas n'est plus lié à une entrée mais devient improbable : l'espérance est $\Theta(n \log n)$ *pour toute entrée*.
- **Médiane de trois** : pivot = médiane de $t[g]$, $t[(g + d)/2]$, $t[d]$. Détruit le pire cas « tableau trié », le plus fréquent en pratique.
- **Bascule** vers le tri par insertion pour $d - g < 15$ (peu d'appels récursifs).

Meilleur cas	$\Theta(n \log n)$ (pivot médian)
Cas moyen	$2(n+1)H_{n+1} - 4n \approx 1,39 n \log_2 n = \Theta(n \log n)$
Pire cas	$\frac{n(n-1)}{2} = \Theta(n^2)$ (tableau déjà trié, pivot extrême)
Mémoire auxiliaire	$\Theta(\log n)$ en moyenne (pile d'appels) — en place
Stabilité	non stable (les échanges à distance cassent l'ordre)

À retenir pour l'oral. « Le tri rapide est en $\Theta(n \log n)$ » est *faux* sans précision. Il est en $\Theta(n \log n)$ **en moyenne**, et en $\Theta(n^2)$ dans le pire cas.

Synthèse

- 1 Le problème du tri : vocabulaire et outils
- 2 Les tris quadratiques : exercices de complexité
- 3 Une borne inférieure : $\Omega(n \log n)$
- 4 Diviser pour régner
- 5 Le tri fusion
- 6 Le tri rapide
- 7 Synthèse**

Tableau comparatif

Tri	Meilleur	Moyen	Pire	Mémoire	En place	Stable
Sélection	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$	oui	non
Insertion	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$	oui	oui
Bulles	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$	oui	oui
Fusion	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$	non	oui
Rapide	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$\Theta(\log n)$	oui	non

Comment choisir ?

- garantie de complexité, stabilité, données volumineuses ou séquentielles → **tri fusion** ;
- vitesse brute sur un tableau en mémoire, mémoire contrainte → **tri rapide** (avec pivot aléatoire ou médian) ;
- n petit (≤ 15) ou tableau presque trié → **tri par insertion**.

n	$\frac{n(n-1)}{2}$	$n \log_2 n$	rapport
10^2	$4,95 \cdot 10^3$	$6,6 \cdot 10^2$	$\times 7$
10^3	$5,0 \cdot 10^5$	$1,0 \cdot 10^4$	$\times 50$
10^4	$5,0 \cdot 10^7$	$1,3 \cdot 10^5$	$\times 376$
10^6	$5,0 \cdot 10^{11}$	$2,0 \cdot 10^7$	$\times 25\,000$

À 10^8 comparaisons par seconde : trier un million d'entiers demande $\approx 1,4$ **heure** avec un tri quadratique, et 0,2 **seconde** avec un tri en $n \log n$. *Le choix de l'algorithme pèse plus lourd que le choix de la machine.*

Les cinq points du chapitre

1. Les trois tris quadratiques coûtent $\Theta(n^2)$ comparaisons dans le pire cas ; seul le tri par insertion est linéaire sur une entrée presque triée (coût = $I(t)$ décalages).
2. Aucun tri par comparaisons ne peut descendre sous $\Omega(n \log n)$: argument de l'arbre de décision, $2^h \geq n!$.
3. *Diviser pour régner* $\Rightarrow$ récurrence $T(n) = aT(n/b) + f(n)$, résolue en déroulant ou par le théorème maître.
4. Tri fusion : $\Theta(n \log n)$ *garanti*, stable, mais $\Theta(n)$ de mémoire.
5. Tri rapide : en place, $\Theta(n \log n)$ *en moyenne* ($\approx 1,39 n \log_2 n$), mais $\Theta(n^2)$ dans le pire cas — d'où le pivot aléatoire.

Et toujours : terminaison (variant / mesure), correction (invariant / récurrence), complexité (récurrence).

1. **Fusion sans recopie.** Écrire `fusion_en_place(t, g, m, d)` utilisant un unique tableau tampon de taille $d - g + 1$. Quelle est la complexité mémoire totale du tri fusion ainsi implanté ?
2. **Comptage d'inversions.** Adapter le tri fusion pour qu'il renvoie $(t_{\text{trié}}, I(t))$ en $\Theta(n \log n)$.
Indication : quand on prend un élément de la moitié droite, combien d'inversions supprime-t-on d'un coup ?
3. **k -ième plus petit élément.** Adapter la partition du tri rapide pour obtenir `selection(t, k)` en $\Theta(n)$ en moyenne (algorithme *quickselect*). Écrire et résoudre la récurrence $C(n) = C(n/2) + n$.
4. **Tri rapide à trois voies.** Partitionner en $\{< p\}$, $\{= p\}$, $\{> p\}$. Quelle est la complexité sur un tableau ne contenant que $\mathcal{O}(1)$ valeurs distinctes ?
5. **Borne fine.** Montrer que $\log_2(n!) = n \log_2 n - n \log_2 e + \Theta(\log n)$; en déduire que le tri fusion est optimal à un terme d'ordre n près.

Fin du chapitre

Algorithmes de tri — CPGE MP / PSI / TSI
